

HERMES AGENT

Documentation officielle en français

Getting Started

Guide de démarrage complet

Editeur	Nous Research
Licence	MIT License — 2026
GitHub	github.com/NousResearch/hermes-agent
Discord	discord.gg/NousResearch

Table des matières

1	Démarrage rapide (Quickstart)	<i>Installation, choix du fournisseur, première conversation</i>
2	Installation	<i>Prérequis, options d'installation, dépannage</i>
3	Android / Termux	<i>Installation sur téléphone Android via Termux</i>
4	Configuration Nix & NixOS	<i>Flake Nix, module NixOS, déploiement serveur</i>
5	Mise à jour & Désinstallation	<i>Mettre à jour, annuler une mise à jour, désinstaller</i>
6	Parcours d'apprentissage	<i>Chemins par niveau et par cas d'utilisation</i>

CHAPITRE 1

Démarrage rapide

De zéro à une configuration fonctionnelle en moins de 5 minutes

À qui s'adresse ce guide ?

Ce guide est destiné aux utilisateurs souhaitant obtenir le chemin le plus court vers une installation fonctionnelle, à ceux qui changent de fournisseur IA, à ceux configurant Hermes pour une équipe ou un bot, ou encore aux utilisateurs frustrés par une installation qui ne fait « rien ».

Le chemin le plus rapide

Choisissez la ligne qui correspond à votre objectif :

Objectif	Première action	Ensuite
Je veux juste qu'Hermes fonctionne sur ma machine	<code>hermes setup</code>	Lancer un vrai chat et vérifier la réponse
Je connais déjà mon fournisseur	<code>hermes model</code>	Sauvegarder la config, puis démarrer le chat
Je veux un bot ou une configuration permanente	<code>hermes gateway setup</code>	Connecter Telegram, Discord, Slack ou autre plateforme
Je veux un modèle local ou auto-hébergé	<code>hermes model → endpoint custom</code>	Vérifier l'endpoint, le nom du modèle et la longueur de contexte
Je veux un basculement multi-fournisseurs	<code>hermes model en premier</code>	Ajouter routage et basculement après que le chat de base fonctionne

- **Règle d'or** : si Hermes ne peut pas compléter un chat normal, n'ajoutez pas encore plus de fonctionnalités. Faites d'abord fonctionner une conversation propre, puis ajoutez passerelle, cron, compétences, voix ou routage.

Étape 1 — Installer Hermes Agent

Exécutez l'installateur en une seule ligne :

```
# Linux / macOS / WSL2 / Android (Termux)
curl -fsSL
https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh |
bash
```

- **Utilisateurs Windows** : installez d'abord WSL2, puis exécutez la commande ci-dessus dans votre terminal WSL2.

Après l'installation, rechargez votre shell :

```
source ~/.bashrc # ou : source ~/.zshrc
```

Étape 2 — Choisir un fournisseur IA

C'est l'étape de configuration la plus importante. Utilisez **hermes model** pour naviguer interactivement :

```
hermes model
```

Fournisseur	Description	Configuration
Nous Portal	Abonnement, zéro config	OAuth via hermes model
Anthropic	Modèles Claude directement	OAuth (plan Max) ou clé API
OpenAI Codex	Auth OAuth ChatGPT	Code de dispositif via hermes model
OpenRouter	Routage multi-fournisseurs	Entrez votre clé API
GitHub Copilot	Abonnement Copilot	OAuth via hermes model
DeepSeek	API DeepSeek directe	Clé DEEPSEEK_API_KEY
AWS Bedrock	Claude, Nova, Llama via API native	Rôle IAM ou aws configure
Custom Endpoint	VLLM, Ollama, API OpenAI-compatible	URL de base + clé API

- **Contexte minimum** : Hermes Agent exige un modèle avec au moins **64 000 tokens** de contexte. Les modèles avec des fenêtres plus petites ne peuvent pas maintenir suffisamment de mémoire de travail pour les workflows multi-étapes et seront rejetés au démarrage.

Stockage des paramètres

Hermes sépare les secrets de la configuration normale :

- Secrets et tokens → `~/.hermes/.env`
- Paramètres non secrets → `~/.hermes/config.yaml`

La manière la plus simple de définir des valeurs est via le CLI :

```
hermes config set model anthropic/claude-opus-4.6
```

```
hermes config set terminal.backend docker
```

```
hermes config set OPENROUTER_API_KEY sk-or-...
```

Étape 3 — Lancer votre premier chat

```
hermes # interface CLI classique
```

```
hermes --tui # interface TUI moderne (recommandée)
```

Exemples de prompts de vérification :

```
Résume ce dépôt en 5 points et dis-moi quel est le point d'entrée principal.
```

```
Vérifie mon répertoire courant et dis-moi quel semble être le fichier de projet principal.
```

Étape 4 — Vérifier les sessions

```
hermes --continue # Reprendre la session la plus récente
hermes -c # Forme abrégée
```

Étape 5 — Tester les fonctionnalités clés

Utiliser le terminal

```
> Quelle est mon utilisation du disque ? Montre les 5 plus grands répertoires.
```

Commandes slash

Tapez / pour voir un menu déroulant d'autocomplétion de toutes les commandes :

Commande	Description
/help	Afficher toutes les commandes disponibles
/tools	Lister les outils disponibles
/model	Changer de modèle interactivement
/personality pirate	Essayer une personnalité amusante
/save	Sauvegarder la conversation

Modes de saisie

- Saisie multi-ligne : appuyez sur **Alt+Entrée** ou **Ctrl+J** pour ajouter une nouvelle ligne.
- Interrompre l'agent : tapez un nouveau message et appuyez sur Entrée, ou utilisez **Ctrl+C**.

Étape 6 — Ajouter des fonctionnalités avancées

Seulement après que le chat de base fonctionne. Voici quelques options :

Bot ou assistant partagé

```
hermes gateway setup
```

Connecter Telegram, Discord, Slack, WhatsApp, Signal, Email, Teams...

Terminal sandboxé

```
hermes config set terminal.backend docker
```

Isolation Docker ou serveur distant via SSH

Mode voix

```
pip install "hermes-agent[voice]"
Puis /voice on dans le CLI
```

Reconnaissance vocale locale avec faster-whisper

Compétences (Skills)

```
hermes skills search <nom>
hermes skills install <skill>
```

Workflows réutilisables installables

Serveurs MCP

```
Configurer dans ~/.hermes/config.yaml
```

Outils externes via Model Context Protocol

Problèmes courants et solutions

Symptôme	Cause probable	Solution
Réponses vides ou cassées	Auth ou sélection de modèle incorrecte	Relancer hermes model
Endpoint custom retourne des données bizarres	URL de base, nom de modèle ou compatibilité OpenAI incorrects	Vérifier l'endpoint séparément
La passerelle démarre mais personne ne peut écrire	Token bot, liste blanche ou config plateforme incomplète	Relancer hermes gateway setup
hermes --continue ne trouve pas l'ancienne session	Profil changé ou session non sauvegardée	Vérifier hermes sessions list

Kit de récupération

1. hermes doctor
2. hermes model
3. hermes setup
4. hermes sessions list
5. hermes --continue
6. hermes gateway status

CHAPITRE 2

Installation

Hermes Agent opérationnel en moins de deux minutes

Installation rapide

Linux / macOS / WSL2

```
curl -fsSL
https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh |
bash
```

Android / Termux

L'installateur détecte automatiquement Termux et bascule vers un flux Android testé :

```
curl -fsSL
https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh |
bash
```

- Utilise **pkg** de Termux pour les dépendances système (git, python, nodejs, ripgrep, ffmpeg, outils de compilation)
- Crée le virtualenv avec **python -m venv**
- Exporte **ANDROID_API_LEVEL** automatiquement pour les builds Android
- Installe le bundle **.[termux]** avec pip
- Ignore le bootstrap navigateur/WhatsApp par défaut

■ **Windows natif non supporté.** Installez WSL2 et exécutez Hermes Agent depuis là.

Ce que fait l'installateur

L'installateur gère tout automatiquement : toutes les dépendances (Python, Node.js, ripgrep, ffmpeg), le clonage du dépôt, l'environnement virtuel, la configuration de la commande globale **hermes** et la configuration du fournisseur LLM. À la fin, vous êtes prêt à discuter.

Emplacement des fichiers installés

Mode	Code source	Binaire hermes	Répertoire de données
Par utilisateur (normal)	~/.hermes/hermes-agent/	~/.local/bin/hermes (lien sym.)	~/.hermes/
Root (sudo)	/usr/local/lib/hermes-agent/	/usr/local/bin/hermes	/root/.hermes/ (ou \$HERMES_HOME)

Après l'installation

```
source ~/.bashrc # ou : source ~/.zshrc
hermes # Démarrer le chat !
```

Pour reconfigurer les paramètres ultérieurement :

```
hermes model # Choisir votre fournisseur LLM et modèle
hermes tools # Configurer les outils activés
hermes gateway setup # Configurer les plateformes de messagerie
hermes config set # Définir des valeurs de config individuelles
hermes setup # Ou lancer l'assistant de configuration complet
```

Prérequis

- Le seul prérequis est **Git**. L'installateur gère automatiquement tout le reste : uv (gestionnaire de paquets Python rapide), Python 3.11 (via uv, sans sudo), Node.js v22 (pour l'automatisation du navigateur et le pont WhatsApp), ripgrep (recherche de fichiers rapide), ffmpeg (conversion de format audio pour TTS).

Dépannage

Problème	Solution
hermes: command not found	Rechargez votre shell (source ~/.bashrc) ou vérifiez le PATH
API key not set	Lancez hermes model pour configurer votre fournisseur, ou hermes config set OPENROUTER_API_KEY votre_cle
Config manquante après mise à jour	Lancez hermes config check puis hermes config migrate

- Pour plus de diagnostics, lancez **hermes doctor** — il vous dira exactement ce qui manque et comment le corriger.

CHAPITRE 3

Android / Termux

Exécuter Hermes Agent directement sur un téléphone Android

Ce chapitre décrit le chemin testé pour exécuter Hermes Agent directement sur un téléphone Android via **Termux**. Il vous donne un CLI local fonctionnel sur le téléphone, ainsi que les extras de base connus pour s'installer proprement sur Android.

Fonctionnalités prises en charge

- Le CLI Hermes
- Support cron
- Support PTY/terminal en arrière-plan
- Support de la passerelle Telegram (exécutions manuelles / best-effort en arrière-plan)
- Support MCP
- Support mémoire Honcho
- Support ACP

Limitations connues sur téléphone

- Le backend Docker n'est pas disponible
- La transcription vocale locale via faster-whisper est indisponible dans le chemin testé
- La configuration automatique du navigateur est intentionnellement ignorée par l'installateur
- Android peut suspendre les tâches en arrière-plan Termux, la persistance de la passerelle est donc en mode best-effort
- `.[all]` n'est pas supporté sur Android aujourd'hui

Option 1 : Installateur en une ligne

```
curl -fsSL
https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh |
bash
```

Sur Termux, l'installateur automatiquement : utilise **pkg** pour les paquets système, crée le venv avec **python -m venv**, installe **.[termux]** avec pip, et lie **hermes** dans **\$PREFIX/bin** pour qu'il reste dans votre PATH Termux.

Option 2 : Installation manuelle (explicite)

1. Mettre à jour Termux et installer les paquets système

```
pkg update
pkg install -y git python clang rust make pkg-config libffi openssl nodejs ripgrep
ffmpeg
```

2. Cloner Hermes

```
git clone --recurse-submodules https://github.com/NousResearch/hermes-agent.git
cd hermes-agent
```

3. Créer un environnement virtuel

```
python -m venv venv
source venv/bin/activate
export ANDROID_API_LEVEL="$(getprop ro.build.version.sdk)"
python -m pip install --upgrade pip setuptools wheel
```

- **ANDROID_API_LEVEL** est important pour les paquets basés sur Rust/maturin tels que jiter.

4. Installer le bundle Termux testé

```
python -m pip install -e '.[termux]' -c constraints-termux.txt
# Ou pour le core minimal seulement :
python -m pip install -e '.' -c constraints-termux.txt
```

5. Ajouter hermes à votre PATH Termux

```
ln -sf "$PWD/venv/bin/hermes" "$PREFIX/bin/hermes"
```

6. Vérifier l'installation

```
hermes version
hermes doctor
```

7. Démarrer Hermes

```
hermes
```

Dépannage Termux

Problème : No solution found lors de l'installation de .[all]

```
Utilisez le bundle Termux :
python -m pip install -e '.[termux]' -c constraints-termux.txt
Le bloqueur est l'extra voice → faster-whisper → ctranslate2 (pas de wheels Android)
```

Problème : uv pip install échoue sur Android

```
Utilisez python -m venv + pip à la place (voir étape 3 ci-dessus)
```

Problème : jiter / maturin se plaint de ANDROID_API_LEVEL

```
Définissez le niveau API :
export ANDROID_API_LEVEL="$(getprop ro.build.version.sdk)"
```

Problème : hermes doctor dit que ripgrep ou Node est manquant

```
pkg install ripgrep nodejs
```

Problème : Échecs de compilation lors de l'installation de paquets Python

```
pkg install clang rust make pkg-config libffi openssl
```

CHAPITRE 4

Configuration Nix & NixOS

Flake Nix, module NixOS natif et mode conteneur

Hermes Agent fournit un flake Nix avec trois niveaux d'intégration :

Niveau	Pour qui	Ce que vous obtenez
nix run / nix profile install	Tout utilisateur Nix (macOS, Linux)	Binaire pré-compilé avec toutes les dépendances, puis workflow CLI standard
Module NixOS (natif)	Déploiements serveur NixOS	Config déclarative, service systemd renforcé, secrets gérés
Module NixOS (conteneur)	Agents nécessitant une auto-modification	Tout ce qui précède + conteneur Ubuntu persistant où l'agent peut apt/pip/npm install

Prérequis

- **Nix avec flakes activés** — Déterminez Nix recommandé (active les flakes par défaut)
- **Clés API** pour les services que vous souhaitez utiliser (au minimum : une clé OpenRouter ou Anthropic)

Démarrage rapide (tout utilisateur Nix)

Aucun clonage nécessaire. Nix récupère, compile et exécute tout :

```
# Exécuter directement (compile à la première utilisation, mis en cache ensuite)
nix run github:NousResearch/hermes-agent -- setup
nix run github:NousResearch/hermes-agent -- chat

# Ou installer de façon persistante
nix profile install github:NousResearch/hermes-agent
hermes setup
hermes chat
```

Module NixOS

- Ce module nécessite NixOS. Pour les systèmes non-NixOS (macOS, autres distributions Linux), utilisez **nix profile install** et le workflow CLI standard.

Ajouter le flake comme entrée

```
# /etc/nixos/flake.nix
{
  inputs = {
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-unstable";
    hermes-agent.url = "github:NousResearch/hermes-agent";
  };
}
```

```

};
outputs = { nixpkgs, hermes-agent, ... }: {
  nixosConfigurations.your-host = nixpkgs.lib.nixosSystem {
    system = "x86_64-linux";
    modules = [
      hermes-agent.nixosModules.default
      ./configuration.nix
    ];
  };
};
};
}

```

Configuration minimale

```

# configuration.nix
{ config, ... }: {
  services.hermes-agent = {
    enable = true;
    settings.model.default = "anthropic/claude-sonnet-4";
    environmentFiles = [ config.sops.secrets."hermes-env".path ];
    addToSystemPackages = true;
  };
}

```

- Un **nixos-rebuild switch** crée l'utilisateur hermes, génère config.yaml, connecte les secrets et démarre la passerelle.

Gestion des secrets

- **Ne jamais mettre les clés API dans settings ou environment.** Les valeurs dans les expressions Nix se retrouvent dans /nix/store, qui est lisible par tous. Utilisez toujours environmentFiles avec un gestionnaire de secrets.

Exemple avec sops-nix :

```

{
  sops = {
    defaultSopsFile = ./secrets/hermes.yaml;
    secrets."hermes-env" = { format = "yaml"; };
  };
  services.hermes-agent.environmentFiles = [
    config.sops.secrets."hermes-env".path
  ];
}

```

Modes de déploiement

	Natif (défaut)	Conteneur
Exécution	Service systemd renforcé sur l'hôte	Conteneur Ubuntu persistant avec /nix/store bind-monté
Sécurité	NoNewPrivileges, ProtectSystem=strict, PrivateTmp	Isolation conteneur, utilisateur non-privilégié à l'intérieur
Auto-install paquets	Non — uniquement outils du PATH Nix	Oui — apt, pip, npm persistent entre redémarrages
Quand choisir	Déploiements standard, sécurité maximale	Agent nécessitant installation de paquets à l'exécution

Mise à jour via Nix

```
# Mettre à jour l'entrée du flake
nix flake update hermes-agent --flake /etc/nixos

# Reconstruire
sudo nixos-rebuild switch
```

CHAPITRE 5

Mise à jour & Désinstallation

Maintenir Hermes Agent à jour et le supprimer si besoin

Mise à jour

Mettez à jour vers la dernière version avec une seule commande :

```
hermes update
```

Ce qui se passe lors d'une mise à jour

- 1. Instantané pré-mise à jour** — Un snapshot léger de l'état est sauvegardé (couvre ~/.hermes/pairing/ et autres fichiers d'état). Récupérable via `hermes backup restore --state pre-update`
- 2. Git pull** — Tire le dernier code depuis la branche main et met à jour les sous-modules
- 3. Installation des dépendances** — Exécute `uv pip install -e ".[all]"` pour récupérer les dépendances nouvelles ou modifiées
- 4. Migration de config** — Détecte les nouvelles options de config ajoutées depuis votre version et vous invite à les définir
- 5. Redémarrage automatique de la passerelle** — Les passerelles en cours d'exécution sont rafraîchies après la mise à jour

Options de mise à jour

Aperçu seulement :

```
hermes update --check
# Vérifie si vous êtes en retard sur origin/main sans modifier aucun fichier
```

Sauvegarde complète avant mise à jour :

```
hermes update --backup
# Ou pour en faire le comportement par défaut dans config.yaml :
# update:
# backup: true
```

Validation post-mise à jour recommandée

- `git status --short` — si l'arbre est inattendu sale, inspectez avant de continuer
- `hermes doctor` — vérifie la config, les dépendances et la santé du service
- `hermes --version` — confirmez que la version a augmenté comme prévu
- Si vous utilisez la passerelle : `hermes gateway status`

Mise à jour depuis les plateformes de messagerie

Vous pouvez également mettre à jour directement depuis Telegram, Discord, Slack ou WhatsApp en envoyant la commande **/update**. Le bot sera brièvement hors ligne pendant le redémarrage (généralement 5-15 secondes).

Mise à jour manuelle

```
cd /chemin/vers/hermes-agent
export VIRTUAL_ENV="$(pwd)/venv"
# Tirer le dernier code et les sous-modules
git pull origin main
git submodule update --init --recursive
# Réinstaller (récupère les nouvelles dépendances)
uv pip install -e "[all]"
# Vérifier les nouvelles options de config
hermes config check
hermes config migrate
```

Instructions de rollback

```
cd /chemin/vers/hermes-agent
# Lister les versions récentes
git log --oneline -10
# Revenir à un commit spécifique
git checkout <hash-du-commit>
git submodule update --init --recursive
uv pip install -e "[all]"
# Redémarrer la passerelle si elle est en cours d'exécution
hermes gateway restart
# Ou revenir à une version étiquetée :
git checkout v0.6.0
```

Désinstallation

L'utilitaire de désinstallation vous donne la possibilité de conserver vos fichiers de configuration (**~/hermes/**) pour une future réinstallation.

```
hermes uninstall
```

Désinstallation manuelle

```
rm -f ~/.local/bin/hermes
rm -rf /chemin/vers/hermes-agent
rm -rf ~/hermes # Optionnel – garder si vous prévoyez de réinstaller
```


- Si vous avez installé la passerelle comme service système, arrêtez-la et désactivez-la d'abord : `hermes gateway stop` # Linux : `systemctl --user disable hermes-gateway` # macOS : `launchctl remove ai.hermes.gateway`

CHAPITRE 6

Parcours d'apprentissage

Par où commencer selon votre niveau et vos objectifs

Hermes Agent peut faire beaucoup de choses : assistant CLI, bot Telegram/Discord, automatisation de tâches, entraînement RL, et bien plus. Cette page vous aide à déterminer par où commencer en fonction de votre niveau d'expérience et de ce que vous souhaitez accomplir.

- Si vous n'avez pas encore installé Hermes Agent, commencez par le **Guide d'installation** (chapitre 2) puis parcourez le **Démarrage rapide** (chapitre 1). Tout ce qui suit suppose une installation fonctionnelle.

Par niveau d'expérience

Niveau	Objectif	Lecture recommandée	Temps estimé
Débutant	Démarrer, avoir des conversations de base, utiliser les outils intégrés	Installation → Quickstart → CLI → Configuration	~1 heure
Intermédiaire	Configurer des bots de messagerie, utiliser mémoire, cron, skills	Sessions → Messagerie → Outils → Skills → Mémoire → Cron	~2-3 heures
Avancé	Créer des outils personnalisés, entraîner des modèles avec RL, contribuer	Architecture → Ajout d'outils → Création de skills → Entraînement RL → Contribution	~4-6 heures

Par cas d'utilisation

■ ■ Je veux un assistant CLI de codage

- Installation → Quickstart → Utilisation CLI
- Exécution de code
- Fichiers de contexte
- Conseils & astuces

- Passez des fichiers directement dans votre conversation avec les fichiers de contexte. Hermes Agent peut lire, éditer et exécuter du code dans vos projets.

■ Je veux un bot Telegram/Discord

- Installation → Configuration
- Vue d'ensemble de la messagerie
- Configuration Telegram / Configuration Discord

- Mode voix → Sécurité

■ Pour des exemples de projets complets, consultez les guides Daily Briefing Bot et Team Telegram Assistant.

■ Je veux automatiser des tâches

- Quickstart
- Planification Cron
- Traitement par lots
- Délégation → Hooks

■ Les tâches cron permettent à Hermes Agent d'exécuter des tâches selon un calendrier : résumés quotidiens, vérifications périodiques, rapports automatisés, sans que vous soyez présent.

■ Je veux créer des outils/compétences personnalisés

- Vue d'ensemble des outils
- Vue d'ensemble des Skills
- MCP (Model Context Protocol)
- Architecture → Ajout d'outils → Création de skills

■ Les outils sont des fonctions individuelles que l'agent peut appeler. Les skills sont des bundles d'outils, de prompts et de configuration packagés ensemble. Commencez par les outils, puis passez aux skills.

■ Je veux entraîner des modèles

- Quickstart → Configuration
- Entraînement RL
- Routage de fournisseurs → Architecture

■ L'entraînement RL fonctionne mieux quand vous comprenez déjà les bases. Parcourez d'abord le chemin Débutant si vous êtes novice.

■ Je veux l'utiliser comme bibliothèque Python

- Installation → Quickstart
- Guide de la bibliothèque Python
- Architecture → Outils → Sessions

■ Intégrez Hermes Agent dans vos propres applications Python de manière programmatique.

Fonctionnalités clés en un coup d'œil

Fonctionnalité	Description
----------------	-------------

Outils (Tools)	Outils intégrés que l'agent peut appeler (E/S fichiers, recherche, shell, etc.)
Compétences (Skills)	Paquets plugins installables qui ajoutent de nouvelles capacités
Mémoire (Memory)	Mémoire persistante entre les sessions
Fichiers de contexte	Alimenter des fichiers et répertoires dans les conversations
MCP	Se connecter à des serveurs d'outils externes via le Model Context Protocol
Cron	Planifier des tâches récurrentes de l'agent
Délégation	Créer des sous-agents pour un travail parallèle
Exécution de code	Exécuter des scripts Python qui appellent les outils Hermes programmatiquement
Navigateur	Navigation web et scraping
Hooks	Callbacks événementiels et middleware
Traitement par lots	Traiter plusieurs entrées en masse
Entraînement RL	Affiner les modèles avec l'apprentissage par renforcement
Routage de fournisseurs	Router les requêtes sur plusieurs fournisseurs LLM

Que lire ensuite ?

Vous venez de finir l'installation ? → Rendez-vous au Démarrage rapide (chapitre 1) pour lancer votre première conversation.

Vous avez terminé le Quickstart ? → Lisez Utilisation du CLI et Configuration pour personnaliser votre installation.

Vous êtes à l'aise avec les bases ? → Explorez Outils, Compétences et Mémoire pour libérer toute la puissance de l'agent.

Vous configurez pour une équipe ? → Lisez Sécurité et Sessions pour comprendre le contrôle d'accès et la gestion des conversations.

Prêt à développer ? → Plongez dans le Guide du développeur pour comprendre les internals et commencer à contribuer.

Vous voulez des exemples pratiques ? → Consultez la section Guides pour des projets réels et des conseils.

- Vous n'avez pas besoin de tout lire. Choisissez le chemin qui correspond à votre objectif, suivez les liens dans l'ordre, et vous serez productif rapidement. Vous pouvez toujours revenir à cette page pour trouver votre prochaine étape.

Hermes Agent — Getting Started

Documentation française traduite depuis la documentation officielle de Nous Research.

Ressources utiles

github.com/NousResearch/hermes-agent | discord.gg/NousResearch | nousresearch.com

MIT License · 2026 · Nous Research